Game Specification

Purpose of This Document

This document is the authoritative specification for the game.

The game must be completed and playable.

Claude Code should read the complete specification before implementation begins.

The highest priority is to produce a complete, playable and enjoyable game. Prefer the simplest implementation that satisfies this specification. Avoid unnecessary architecture, dependencies, abstractions or features.

A playable end-to-end version of the core game should be established as early as possible.

All **Must Have** requirements must be completed before any **Stretch** requirements are implemented.

Where practical, game content must be separated from game logic so that other team members can continue creating and refining content while development is underway.

The game should be run and checked regularly throughout development. Integration and testing should not be left until the end.

Where this specification is ambiguous, choose the simplest reasonable interpretation that is consistent with the rest of the document.

---

1. Game Concept

Working Title

[Enter working title]

One-Sentence Pitch

Keep your product alive for as long as possible by balancing people, quality, deadlines and budget while coping with unreasonable stakeholders and constant disasters.

Player Role

The player is a product person responsible for keeping their product alive as long as possible.

Player Objective

The player needs to keep the product alive for as long as possible.

Core Gameplay Loop

The player gets presented with a random card. The player has two choices to how to respond to the card. They can swipe left for one choice or swipe right for the alternative choice. Every choice increases or decreases one or more of the products team, quality, deadlines or budget metrics. They are then presented with another card. Each new card advances the product lifetime clock by one month. This continues until one of the metrics hits zero or 100. If a metric hits zero or 100 it is game over. The player aims to keep the product alive for as many months as possible.

Game Length

5-15 minutes

Tone

The tone should be funny and chaotic. The cards that get presented will be hyperbolic, cartoonified versions of the real issues that get presented in real product development. For example, instead of a cloud provider outage an escaped hippo from the zoo has taken up residence in a data center and has eaten the products staging server. The gameplay should encourage rapid playthroughs rather than complex deliberation. Each choice will have a damned if you do, damned if you don't feel.

What Makes It Fun?

Each playthrough will be quick, different and random. It will create a just another run mentality to see if you can get to launch. Sometimes a great looking run will be derailed by a random inescapable situation. That will be frustrating but encourage players to want another go.

---

2. Gameplay and Rules

Player Actions

Describe everything the player can do.

Action	Input	Description
Swipe left or right on an	Left for one choice, right for	Each choice leads to an

Action	Input	Description
issue card	the other.	increase or decrease in the product metrics
As a player swipes right or left they see a preview of the impact the choice will have on their four metrics.	If the player swipes left or right but does not let go they see the preview, if they let go the effect takes place	This allows the player to manage the impact of their choices on their metrics.

Game Rules

Describe the rules in clear, plain English.

1. The player must always make one of the choices when presented with a card
2. Every choice leads to an increase or decrease in one or more metrics
3. When any of the four metrics hits zero or 100 the game is over
4. Each card navigated without a metric hitting zero advances the product lifespan by 1 month
5. The longer a player can keep a product alive the more successful the run
6. The player sees a different end card based on how long they keep the product alive
7. Cards can only appear once in a run, once all cards have been exhausted the player is considered successful and is presented with an end card.

Starting State

The Team, Quality, Deadline and Budget metrics are all half full. The product lifetime clock starts at 0 months.

Scoring / Success

Each choice on a card leads to an increase or decrease in the metrics. Each metric has a max value of 100 and a min value of 0. Each metric starts on 50. Any choice on a card can have one of these effects on a metric:

0 effect – the choice does not change the metric

Small decrease – the metric decreases by 10

Large decrease – the metric decreases by 20

Small increase – the metric increases by 10

Large increase - the metric increases by 20

There are separate game over cards for failing on each metric

Team 0 – unhappy team card

Team 100 – complacent team card

Quality 0 – product fails card

Quality 100 – product too good card

Deadline 0 – product outsourced card

Deadline 100 – product rushed card

Budget 0 – run out of money card

Budget 100 – financial impropriety card

Each card successfully navigated advances the product lifetime clock by 1 month

The lifetime for a product determines what end card you see. This card will appear after the game over card unless the player exhausts the deck then they will see this card without seeing the game over card:

0-6 months – end card 1

7-12 months – end card 2

13-18 months – end card 3

19 – 24 months – end card 4

25 months + - end card 5

Failure / Game Over

Any metric that hits zero results in game over.

Progression and Difficulty

The cards are presented randomly so there is no increase in difficulty. What could be an easy card in one run could be impossible if presented at a different point of a subsequent run.

Randomness

> We will have a set of cards that could be presented to a player. These cards should be selected at random from the deck each go. So every run through is different in terms of cards and their order. Players will get familiar with cards over time and use that knowledge to manage runs better.

Restart / Replay

This is a roguelike deck game. Each run is short and the gameplay should encourage the player to try again immediately.

---

3. Game Content

Game content should be kept separate from application logic wherever practical.

This section does **not** need to contain every piece of finished game content before development begins.

Once the format for one unit of content has been agreed, development can begin while team members continue producing additional content in parallel.

Content Type

This is a card based game. Each piece of content is an individual card that describes a situation and asks the player to make one of two choices. The writing on the card should be brief

Content Fields

Define the information required for each content item.

Field	Description	Required?	Example
ID	Unique identifier	Yes	
Card type	Choose from one of the options Start card Regular draw Game over card End card	Yes	
Situation text	A description of the situation the player needs to respond to	Yes	
Situation illustration	An illustration to accompany the situation text	Yes	
Swipe left text	A description of the choice the player makes	No	

Field	Description	Required?	Example
	by swiping left		
Swipe left effect	The change to the metrics that will happen if the player swipes left	No	
Swipe right text	A description of the choice the player makes by swiping right	No	
Swipe right effect	The change to the metrics that will happen if the player swipes right	No	

Where practical, each independently authored content item should be representable as a single row in the team's shared content spreadsheet.

Initial Content

Provide enough finished content here to allow development of a playable version of the game.

Content will be drafted in this spreadsheet: Reigns product - content spreadsheet.xlsx

Additional Content

Additional content may continue to be produced during development using the agreed content format.

---

4. Visual and Audio Design

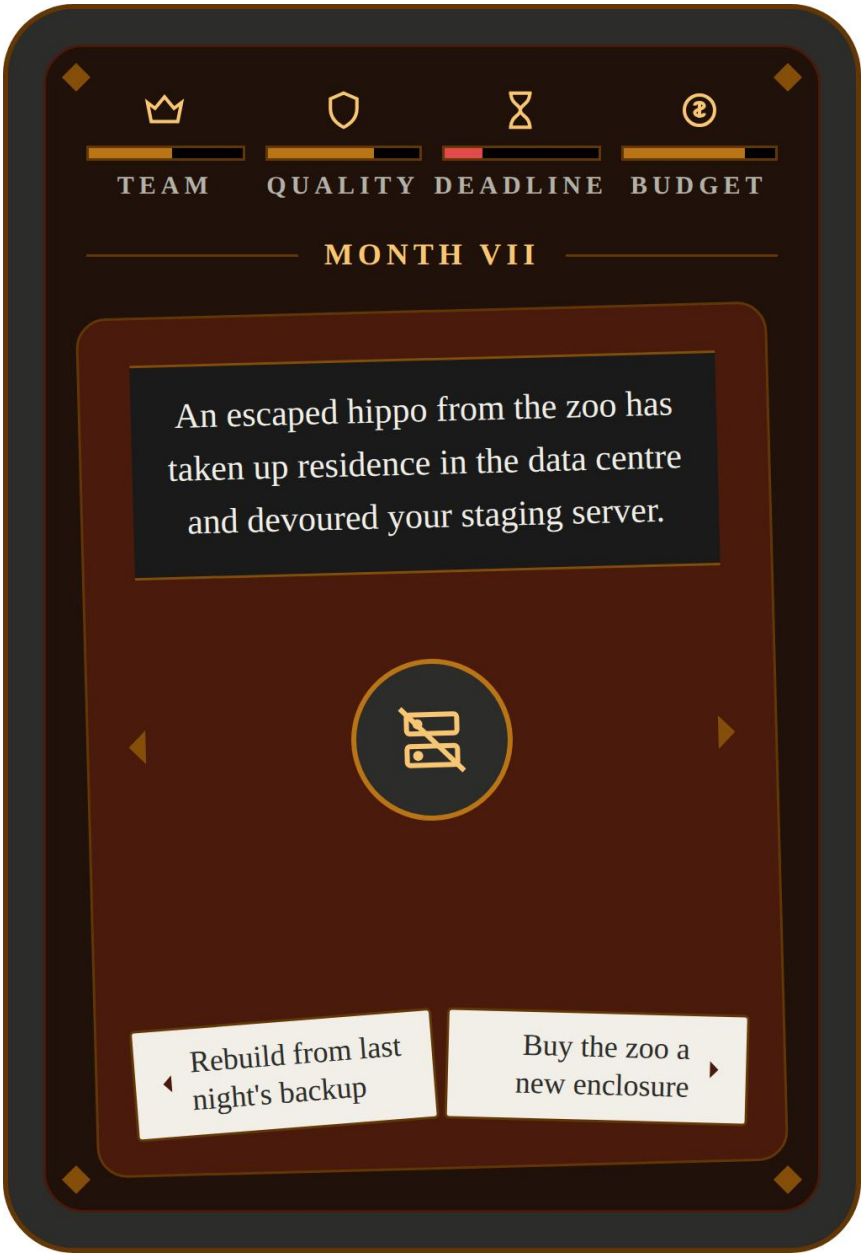
Overall Visual Style

Minimalist, gothic and gloomy. Use the video game Reigns as a reference point.

Every card should have an illustration. Each illustration should follow the same visual style. It should be minimalist, cartoony and gothic just like the overall feel for the game. Avoid too much detail, prefer a simple icon or cartoon wherever possible. When illustrating people or creatures, just represent their head to make the most of available space. Prefer recognizability over detail or delivering the illustration description in complete fidelity. You should use a full colour palette for the illustrations, you do not need to stick to a limited palette. Ensure all colours used fit the overall theme.

Main Game Screen

Paste a wireframe, sketch or mock-up below.



Label important elements where necessary.

Other Screens

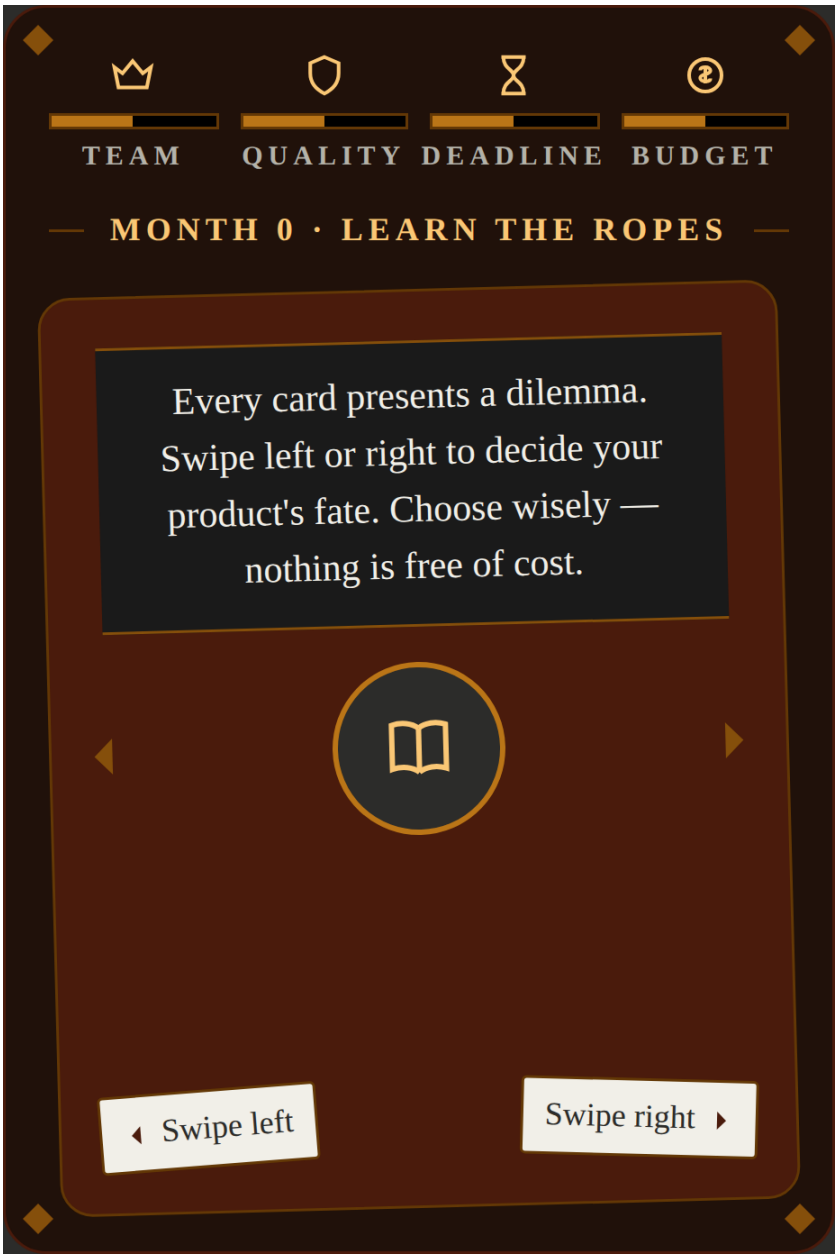
Only include screens that are actually required.

Start screen first playthrough

Purpose:
Teach the player how to play the game

Content:
Intro to the game purpose and a tutorial on how to play the game

Player interactions:
Swipe left and right to get used to the gameplay mechanic



Start screen subsequent playthroughs

Purpose:
Start the player off on a new run

Content:
Intro to the new run

Player interactions:
Swipe left and right to start a new run

Game over screen

Purpose:
End a run by explaining to the player that they have failed

Content:
A description of why they have failed

Player interactions:
Swipe left or right to exit the current run and start a new one

End screen

Purpose:
Congratulate or insult the player on how long their product was kept alive

Content:
A description of the length of the product lifetime

Player interactions:
Swipe left or right to end the current run and start a new run

UI

Describe important interface elements.

- Product lifetime clock – this should display how many months the product has been alive for
- Product metrics – visual representations of the health of the products team, quality, deadlines and budget
- Swipe left and right preview – a preview of what will happen if you swipe left or swipe right
- Situation card

Colours and Typography

Refer to mock ups

Animation and Visual Feedback

Describe important visual feedback.

The card follows the pointer/finger with a slight rotation proportional to horizontal drag distance; if released past a threshold it flies off in the swipe direction, otherwise it snaps back to center

Changes to the metrics should be highlighted after every swipe with a visible change to the metric.

Audio

Music

Music is a nice to have

Sound Effects

Sound effects are a nice to have

Event	Sound / Style

5. Technical Specification

Technology Stack

The game will use:

- **Language:** TypeScript
- **Game / rendering framework:** PixiJS 8
- **Development and build tooling:** Vite
- **Package manager:** pnpm
- **Runtime game content:** JSON
- **Team content authoring:** Microsoft Excel / shared spreadsheet
- **Styling outside the PixiJS canvas:** Plain CSS where required
- **Persistence:** Browser `localStorage` only if required
- **Target:** Modern desktop web browsers

The game must be completely client-side.

Do not introduce a backend, database or runtime network service unless explicitly required elsewhere in this specification.

Do not introduce React, Vue, an ECS framework, a state-management framework or other major architectural dependency unless there is a clear requirement for it. If there is, use Svelte and keep it clean and simple.

Development Priorities

Prioritise:

1. A complete playable gameplay loop.
2. Correct implementation of the core mechanic.
3. Easy integration of team-created content.
4. Clear and responsive player feedback.
5. Visual and audio polish.
6. Stretch functionality.

Do not prioritise architectural sophistication over completing the game.

Content Architecture

Game content must be separated from game logic wherever practical.

Non-technical team members will create and edit content in a shared spreadsheet.

Where practical:

One independently authored content item should correspond to one spreadsheet row.

The spreadsheet content should be exported as CSV and converted into validated JSON for use by the game.

The exact content schema must be based on the **Game Content** section of this specification.

Gameplay balancing values should also be kept outside the core application logic wherever practical so that they can be adjusted quickly.

Content Validation

Externally authored content must be validated before it is consumed by the game.

Validation should include all relevant checks, including:

- required fields are present;
- IDs are unique;
- values use the correct data types;
- permitted values are valid;
- numeric values fall within sensible ranges;
- references to other content IDs are valid;
- referenced assets exist;
- required text is not empty;
- malformed content cannot cause the game to crash;
- any additional game-specific constraints are satisfied.

Validation errors should clearly identify the affected content item or spreadsheet row and explain the problem.

Project Structure

Use a simple, understandable project structure.

A suitable starting structure is:

```
src/
  main.ts
  game/
  scenes/
  systems/
  ui/
  types/
  utils/
```

```
public/
  assets/
    images/
    audio/
  data/

content/
  game-content.csv

scripts/
  build-content.ts
```

Adapt this where necessary for the chosen game.

Build and Quality Checks

Provide the following commands:

`pnpm run dev`

Runs the game locally for development.

`pnpm run build`

Creates the production build.

`pnpm run check`

Runs the project's automated quality checks.

`pnpm run check` should, at minimum:

1. validate game content;
2. run TypeScript type checking;
3. verify that the production build succeeds.

A broad automated unit-test suite is **not required** unless the chosen game contains complex standalone logic that clearly benefits from automated tests.

Gameplay, visuals, controls and overall experience should primarily be verified through manual play-testing.

---

6. Scope and Definition of Done

Must Have

These requirements define the minimum complete game.

The game is not finished until every Must Have requirement works.

- ☐ There is an intro that describes how to play the game
- ☐ Players can swipe left and right to make choices
- ☐ The metrics are updated when a choice requires it
- ☐ The game ends when a metric hits 0 or 100 and a player is served a game over screen
- ☐ The game ends when there are no more cards that can be dealt
- ☐ After the game ends the player is served a card corresponding to how long they kept the product alive. This should report the months the product was kept alive and contain a message that changes depending on how long they kept the product alive.
- ☐ After the player is served the end card they have an option to start another run.
- ☐ Starting another run triggers the display of a start playthrough card that is different to the intro card.

Stretch

Only implement these after all Must Have functionality is complete and the full game has been successfully played from beginning to end.

- ☐ Background music plays during gameplay
- ☐ Sound effects play on swipe and metric change
- ☐ [Stretch feature]

Definition of Done

The game is complete when:

- ☐ It launches successfully.
- ☐ A new player can understand how to play.
- ☐ The complete core gameplay loop works.
- ☐ Win, success, failure or game-over behaviour works as specified.
- ☐ The player can complete a full playthrough.
- ☐ The player can restart or play again without manually refreshing or restarting the application.
- ☐ Team-created content loads correctly.
- ☐ Invalid content is detected by the validation process.
- ☐ `npm run check` completes successfully.
- ☐ There are no known errors that prevent normal gameplay.
- ☐ The game has been play-tested by at least one team member other than the developer.
- ☐ All Must Have requirements are complete.

Final Play-Test

Before development is considered finished, play through the game as a new player would:

> Launch → Understand what to do → Start → Play the complete gameplay loop → Reach the ending / game over → Play again

Fix issues that prevent or significantly harm this experience before implementing additional Stretch functionality.